

Generics

Advanced C# Programming - NET .022

Prepared By: Rupinder Sandhu
Rupinder.sandhu@humber.ca



**WE ARE
HUMBER**

Agenda

- Course Format and Content
- Useful Resources
- Generics

Course Format and Content

- Review of curriculum
 - Tuesday's and Thursday's 6 pm to 9:30 pm
 - Saturday's 9 am to 1 pm

Marks

• Assignments (2 assignments)	40%
• Participation	10%
• Test	20%
• Final Exam	30%
• Total	100%

Useful Resources

- Microsoft Developer Network msdn.microsoft.com
- .NET Fiddle dotnetfiddle.net/
- Stack Overflow stackoverflow.com
- Microsoft Docs <https://docs.microsoft.com/en-gb/>

Generics Agenda

- C# without Generics
- Nullable Modifier
- Generic Types
- Constraints
- Generic Methods
- Generic Interfaces

Introduction to Generics

- Introduced in .NET 2.0.
- Why Generics?
 - Type Safety
 - Compile time checking
 - Type-safe data structures
 - Avoids boxing and unboxing
 - Write code for different types with the same behaviour
 - Reuse data processing algorithms without duplicating type-specific code
 - E.g. `Dictionary<T,T>` can be `Dictionary<string, int>` or `Dictionary<string, double>`

With and Without Generics

- **Without** example code

- Arrays: Provide type safety but are not flexible

- <https://dotnetfiddle.net/Xt8TDf>

- Object List: Flexible but not type safe

- <https://dotnetfiddle.net/sFJOrR>

- Hand Crafted List: Type safe but very cumbersome to create and maintain

- <https://dotnetfiddle.net/tWC1C1>

- **With** example code

- Generic List

- <https://dotnetfiddle.net/8WuWzR>

Boxing and Unboxing

- Boxing is the process of converting a value type to the type object.
- When the CLR boxes a value type, it wraps the value type inside a System. Object and stores it on the managed heap.
- Unboxing extracts the value type from the object. Boxing is implicit; unboxing is explicit.

Code Reuse

- Generics allow a program to use the same code on multiple types of data
- Reduce the amount of code that a programmer has to change to maintain an application

Generic Type Parameters

- In a generic type or method definition, you specify the type of object that the collection **MUST** have (hence, type-safe). A generic item has a pair of angled brackets with a letter or other token within it.
 - For example 'GenericClass<T>'
- Client code must declare and instantiate a constructed type by specifying a type argument inside the angle brackets.
 - For example 'new List<int>()'
 - For example 'new Dictionary<string, string>()'

Type Parameter Naming Guidelines

- **Do** name generic type parameters with descriptive names, unless a single letter name is completely self explanatory and a descriptive name would not add value.
- **Consider** using T as the type parameter name for types with one single type parameter.
- **Do** prefix descriptive type parameter names with "T".
- **Consider** indicating constraints placed on a type parameter in the name of parameter. (we'll look at constraints later)

Generic Classes

- Implementing Generics
 - Generic class = normal class with the generic type declaration `<T>`
 - Place the type parameter in angle brackets
- Generic Features
 - Default Values
 - Constraints
 - Inheritance
 - Static Members
 - Interfaces
- [5. Generics - Creating your own generic class](#)

Default Values

- Types are not known at design time so a developer cannot assign a value to a generic type
- C# gets around this by providing the default keyword
- <https://dotnetfiddle.net/ohIPYN>

Generic Constraints

When you define a generic class, you can apply restrictions to the kinds of types that client code can use for type arguments when it instantiates your class.

- Ensure that the type parameter provides certain methods or properties
- Limit the type parameters of a generic class to those that implement a particular set of attributes
- E.g. `DocumentManager<TDocument>` where `TDocument : IDocument`
- <https://dotnetfiddle.net/GFQXkp>

Generic Constraints (cont.)

Constraint	Description
where T: struct	With a struct constraint, type T must be a value type .
where T: class	The class constraint indicates that type T must be a reference type .
where T: IFoo	Specifies that type T is required to implement an interface .
where T: Foo	Specifies that type T is required to derive from a base class .
where T: new()	A constructor constraint; specifies that type T must have a default (parameterless) constructor .
where T1: T2	With constraints it is also possible to specify that type T1 derives from a generic type T2 .

Generic Constraints (cont.)

- Multiple constraints can be applied to the same type parameter, and the constraints themselves can be generic types
- Constraint Limitations
 - Operator Constraints Are Not Allowed
 - OR Criteria Are Not Supported
 - If you supply multiple interfaces or class constraints for a type parameter, the compiler always assumes an AND relationship between constraints.
- <https://dotnetfiddle.net/r5gJ6z>

Generic Static Members

- Static members are only shared with one instantiation of the class

– E.g.

- `myclass<string>.property == myclass<string>.property`
- `myclass<string>.property != myclass<int>.property`

<https://dotnetfiddle.net/CqsWFP>

Generic Interfaces

- Interfaces are a contract in code
 - Guarantees a class with have certain attributes
- Interfaces can be generic the same way classes and structures can be generic
- <https://dotnetfiddle.net/00NDHn>

Generic Methods

- Generic methods can be defined using the <T> notation
- Generic methods can be defined within non-generic classes
- Constraints
- Specialization
 - Generic methods can be overloaded
- <https://dotnetfiddle.net/kK2a9B>

THANK YOU.



WE ARE

HUMBER